SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I __saravanan.t___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

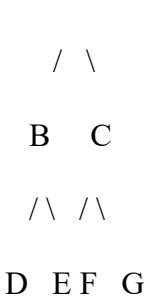
Signature of the student:
SARAVANAN.T

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



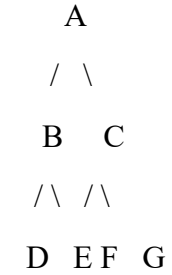
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

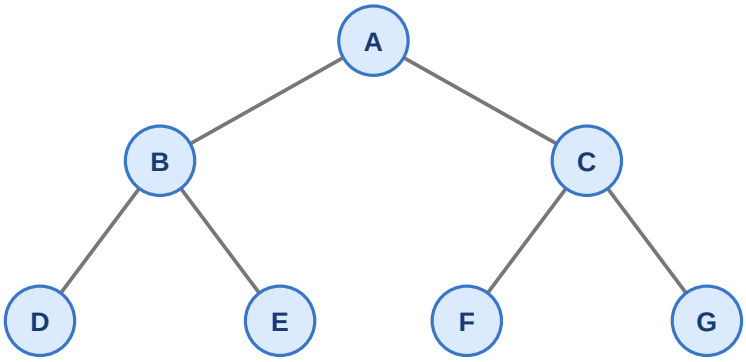
1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1 – Breadth-First Search (BFS)

Definition: Breadth-First Search (BFS) visits nodes level by level. It uses a queue that follows FIFO (First In, First Out).



Assumption: Nodes are processed from left to right.

Queue and Visited Nodes Table

Iteration	Queue (after visiting)	Visited Node
1	[B, C]	A
2	[C, D, E]	B
3	[D, E, F, G]	C
4	[E, F, G]	D
5	[F, G]	E
6	[G]	F
7	[] (Empty)	G

1. BFS Traversal Order

A → B → C → D → E → F → G

2. Queue Contents After Each Iteration

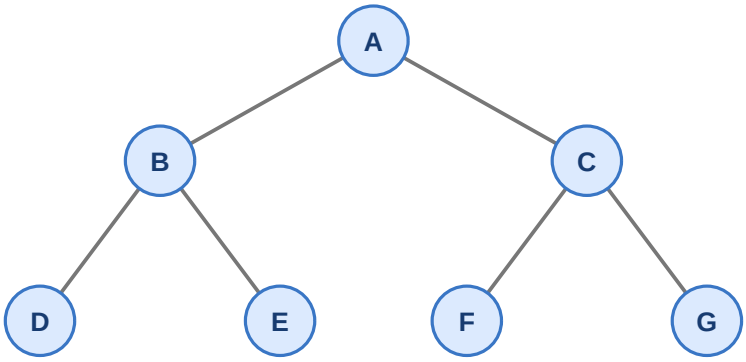
1. Visit A: Queue = [B, C]
2. Visit B: Queue = [C, D, E]
3. Visit C: Queue = [D, E, F, G]
4. Visit D: Queue = [E, F, G]
5. Visit E: Queue = [F, G]
6. Visit F: Queue = [G]

7. Visit G: Queue = [] (empty)

Conclusion: BFS visits every node level by level, using a FIFO queue.

Depth-First Search (DFS) – Solution

Definition: Depth-First Search (DFS) explores as far as possible along one branch before backtracking. Assume children are visited from left to right.



Assumption: Visit the left child before the right child.

DFS Table (Recursive / Preorder Traversal)

Iteration	Current Node	Action / Path
1	A	Start at A
2	B	Go left from A to B
3	D	Go left from B to D
4	E	Backtrack to B; visit E
5	C	Backtrack to A; go right to C
6	F	Go left from C to F
7	G	Backtrack to C; visit G

DFS Traversal Order

A → B → D → E → C → F → G

Step-by-step explanation

1. Start at A.
2. Move to B, then go deeper to D.
3. D has no unvisited children, so backtrack to B and visit E.
4. Backtrack to A and move to C.
5. Visit F, then backtrack to C and visit G.

Note: DFS does not use a FIFO queue like BFS. It is commonly implemented using recursion or a stack.

8-Puzzle Problem — Breadth-First Search (BFS)

Rule used: BFS explores states level by level. The blank (0) moves in this order: Up, Down, Left, Right. States are written as 9 digits in row-major order; 0 represents the blank.

Initial state: 136502478 **Goal state:** 123456780

First five iterations (queue shown before and after removing/expanding the current state)

Iteration	Current state (3×3)	Queue before expansion	Queue after expansion
1	1 3 6 5 0 2 4 7 8	[136502478]	[106532478, 136572408, 136052478, 136520478]
2	1 0 6 5 3 2 4 7 8	[106532478, 136572408, 136052478, 136520478]	[136572408, 136052478, 136520478, 016532478, 160532478]
3	1 3 6 5 7 2 4 0 8	[136572408, 136052478, 136520478, 016532478, 160532478]	[136052478, 136520478, 016532478, 160532478, 136572048, 136572480]
4	1 3 6 0 5 2 4 7 8	[136052478, 136520478, 016532478, 160532478, 136572048, 136572480]	[136520478, 016532478, 160532478, 136572048, 136572480, 036152478, 136452078]
5	1 3 6 5 2 0 4 7 8	[136520478, 016532478, 160532478, 136572048, 136572480, 036152478, 136452078]	[016532478, 160532478, 136572048, 136572480, 036152478, 136452078, 130526478, 136528470]

Important: The goal is not reached within the first five expansions. The table above shows the first five iterations; BFS continues beyond them.

1. Which node/state is expanded in each iteration?

Iterations 1–5 expand, in order: 136502478 → 106532478 → 136572408 → 136052478 → 136520478.

2. How many states are generated in total?

With the stated move order and counting unique discovered states up to the point the goal is removed from the queue, **500 unique states** have been discovered (including the initial state and goal). Counts can vary if a different neighbor order or counting convention is used.

3. Complete shortest solution path

Step 0 (Start): 1 3 6
5 0 2
4 7 8

Step 1 (Right): 1 3 6
5 2 0
4 7 8

Step 2 (Up): 1 3 0
5 2 6

4 7 8

Step 3 (Left): 1 0 3
5 2 6
4 7 8

Step 4 (Down): 1 2 3
5 0 6
4 7 8

Step 5 (Left): 1 2 3
0 5 6
4 7 8

Step 6 (Down): 1 2 3
4 5 6
0 7 8

Step 7 (Right): 1 2 3
4 5 6
7 0 8

Step 8 (Right): 1 2 3
4 5 6
7 8 0

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

Each legal move has equal cost (one move). BFS explores all states at depth 0, then depth 1, then depth 2, and so on. Therefore, the first time it reaches the goal, it has found a path with the fewest possible moves.

Shortest solution: 8 moves — Right, Up, Left, Down, Left, Down, Right, Right.